arcon

INTERNAL · IMPLEMENTATION

Connector Information Gathering Tool
Technical review

A plain-language picture of the system that is already built: who uses it, the technical stack, the folder structure, the use cases, and the decisions to make before a larger UML exercise.

PRODUCT
CIGT

DATE
23 October 2026

AUDIENCE
CTO and technical leadership

STATUS
Working internal application. Suitable for demo and technical review. Not ready for customer production data yet.

1. Read this first

This document explains the application that exists today. It is the shared picture to agree on before the team draws formal use-case diagrams, class diagrams, or a bigger future scope.

CIGT stands for Connector Information Gathering Tool. It is an internal ARCON web application. Implementation engineers use it instead of the Word document they used to fill in when ARCON builds a connector for a customer application.

> **What to take from this pack**
>
> The product shape is already chosen and working: a website, one API, SQL Server for data, a file folder for PDFs and images, and PDF as the only export. The use cases in section 4 are the baseline. New scope should be added to that list. It should keep this stack.

IN ONE SENTENCE

An engineer records how ARCON will connect to a customer application, attaches proof, and produces a branded PDF for the connector team.

WHO IT IS FOR

Implementation engineers, an admin, and a customer who fills a public link. The connector team receives an Outlook draft. They do not log in.

WHAT IT IS NOT

It is not a high-traffic public product. It does not send email itself. It does not store screenshots inside the database. Word and PowerPoint export were removed on purpose.

The job, in order

1. An engineer signs in with an `@arconnet.com` account.
2. They create a **client** (the customer company) and an **application** under that client (GitLab, Workday, SAP, and so on).
3. They fill a structured questionnaire: purpose, how we connect, user lifecycle, SSO, source of truth, encryption, and comments.
4. They attach supporting files: PDF, PNG, or JPG.
5. A customer can fill the same kind of form from a private link, with no ARCON login.
6. The system builds one branded PDF that includes the answers and the uploaded files.
7. The engineer downloads that PDF, or downloads an Outlook draft (an `.eml` file) addressed to the connector team, with the PDF already attached.
8. A later edit is saved as a new version. The previous version stays available.

2. The people

Four kinds of person matter. Three of them have a login role. The customer and the connector team do not.

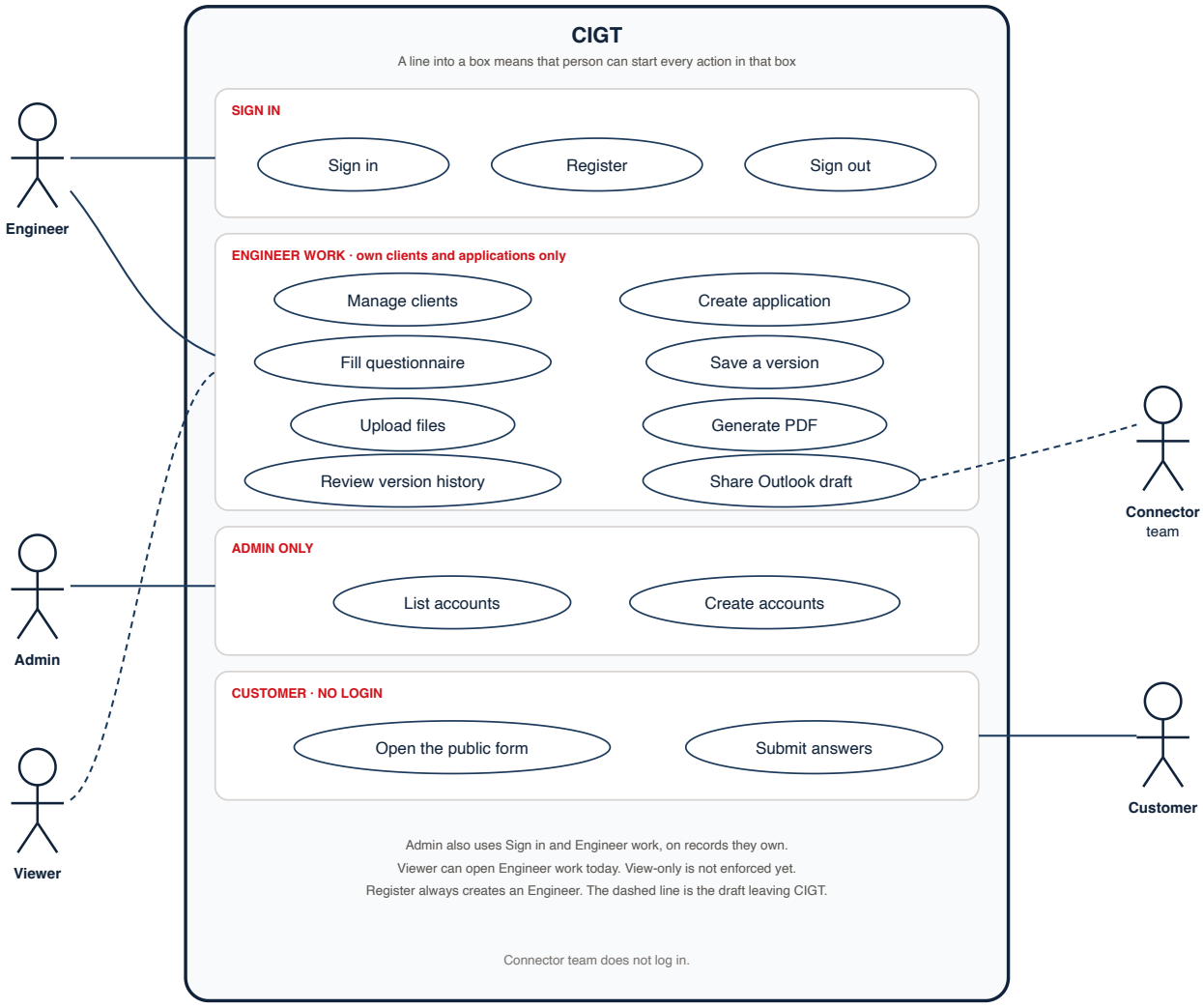
Person	How they get in	What they can do today
Implementation Engineer	Signs in. Anyone who registers on the public page is always created as an Engineer. The page ignores any other role they might send.	Their own clients, applications, questionnaire, files, PDF, and share. They do not see another engineer's clients or applications.
Admin	A seeded account, or an account created by an existing Admin on the Users page.	The same product screens for their own clients and applications, plus a Users page: list accounts and create an Engineer, a Viewer, or another Admin.
Viewer	Created only by an Admin. There is no public way to become a Viewer.	The role exists. The screens are the same as an Engineer's. The system does not yet block a Viewer from editing or deleting their own records. That is a go-live gap, not a finished permission model.
Customer	A link of the form `/form/{token}`. No account.	Reads and submits the requirement form for one application. The token is an unguessable id. Anyone who has the link can use it.
Connector team	No login to CIGT.	They are the people the Outlook draft is addressed to. The engineer opens the draft in Outlook and presses Send. The server does not send the mail.

> **A point that is easy to miss.** Admin does not mean "sees every engineer's work." Clients and applications are stored with the id of the person who created them. Each signed-in user sees only their own. Admin's extra power today is the Users page.

3. Use case diagram

This is the current system boundary. A line means that person can start that action. The connector team sits outside the system: they receive a file, they do not use the website.

Formal UML can redraw this later. The names and the boundary should stay. New scope should appear as a new oval, not as a second product.



Engineer reaches Sign in and Engineer work. Admin reaches Admin only, and also uses Sign in and Engineer work on their own records. Viewer is dashed into Engineer work because those screens are open to them today. Customer reaches the public form. The dashed line on the right is the Outlook draft going to the connector team.

4. Use case catalogue

Each row is one use case a later UML model can keep. The wording is the behaviour of the code today.

Id	Who starts it	What happens
UC-01 Sign in	Engineer, Admin, Viewer	Email and password. The email must end with @arconnet.com. Five failures lock the account for 15 minutes. The browser receives a short-lived access token and a refresh token.
UC-02 Register	New ARCON person	Creates an account. The role is always Engineer, even if the request asks for Admin or Viewer.
UC-03 Sign out	Signed-in user	The refresh token for that session is revoked.
UC-04 See the dashboard	Signed-in user	Search and open the clients and applications that this user owns.
UC-05 Manage clients	Signed-in user	Create a customer company, open it, or delete it. Deleting a client also deletes that user's applications for that client.
UC-06 Create an application	Signed-in user	Creates a project under a client: name, priority, status, engineer, expected date. A public form token can be generated for it.
UC-07 Fill the questionnaire	Engineer	Seven sections, listed in section 11. Answers live in the browser until they are saved.
UC-08 Save a version	Engineer	Writes a document version in SQL Server: the answers, the attachment list, and a completion percent. That saved row is the source of truth.
UC-09 Upload a file	Engineer	The bytes go to a folder on the API machine. SQL Server stores the file name, type, size, and path. The PDF step understands PDF, PNG, JPG, and JPEG.
UC-10 Generate the PDF	Engineer	Builds the ARCON-branded body, appends uploaded PDFs, and places images on their own pages. A missing file is skipped. The rest of the PDF is still produced.
UC-11 Download the PDF	Engineer	The browser downloads the latest generated PDF.
UC-12 Share	Engineer	The browser downloads an Outlook .eml with the PDF attached, addressed to the connector team. The server does not send it.
UC-13 Review history	Engineer	Opens earlier versions. A later edit becomes a new version. The old one remains.
UC-14 Open the public form	Customer	No login. The link identifies one application.
UC-15 Submit the public form	Customer	Stores the customer's answers against that application.
UC-16 List accounts	Admin	Users page. Any other role receives "not allowed."
UC-17 Create an account	Admin	Creates an Engineer, Viewer, or Admin. This is the only way to create a Viewer or an Admin.

Id	Who starts it	What happens
UC-18 Profile and password	Signed-in user	The person can read their profile and change their password.

> **Screens that look finished and are not.** “Verify email” accepts any non-empty code. “Forgot password” creates a reset token and does not email it. Login does not require a confirmed email. The server has no mail connection. Do not describe those two screens as working email.

5. The main journey

This is the path a demo and a real project both follow.

```
Engineer signs in
        |
        v
Dashboard  —  search the clients and applications this person owns
        |
        v
Add a client  —  company record in SQL Server, owned by this user
        |
        v
Add an application  —  project row, plus a public form token
        |
        +—— Engineer fills the workspace
        |      (unsaved typing stays in that browser only)
        |
        +—— Customer opens /form/{token} and submits
        |
        v
Save  —  a new document version (answers + file list)
        |
        v
Upload PDF or image  —  file on disk, details in the database
        |
        v
Generate PDF  —  branded body + merged files
        |
        +—— Download the PDF
        |
        +—— Share  —  browser downloads an Outlook draft
                        Engineer opens it and presses Send
```

Where each piece of information lives

Information	Where it lives	What that means
Unsaved typing in the form	That browser only (local storage)	A cache. Changing laptops before Save loses it. Clearing the site data loses it.
Saved answers and versions	SQL Server	The source of truth after Save.
File name, size, type, path	SQL Server, Attachments table	The catalogue of files. Not the file itself.

Information	Where it lives	What that means
The PDF and the images	A folder on the API machine	The bytes. A database backup does not include them.
The login pass	The browser, as a token	The access token lasts 60 minutes in the production setting (120 in development). The refresh token lasts 7 days.

6. Technical stack

These choices are already in the code. A later design workshop should treat them as fixed unless the CTO explicitly changes one of them.

Layer	What we use	Why it is there
Website	Angular 20, TypeScript, standalone components	The screens. Each screen is loaded when the person opens it.
Look and forms	Angular Material, SCSS, Reactive Forms, Signals	ARCON UI, validated forms, and simple page state. There is no second state library.
API	ASP.NET Core 10, C#	The only program that talks to the database and the file folder.
Shape of the API	Controller, then Service, then Repository	The controller receives HTTP. The service holds the rules. The repository runs the query. Every response uses one envelope: success, message, and data.
Login	ASP.NET Core Identity, JWT access token, refresh tokens	Company accounts. Emails must end with @arconnet.com.
Database	Microsoft SQL Server 2022, Entity Framework Core 10	Tables and migrations. Development runs SQL Server in Docker, database name `cigt`. Production is expected to be a real SQL Server. MySQL, Postgres, and SQLite are out of scope.
PDF body	QuestPDF 2025.7	The branded document: logo, sections, answers.
PDF merge	PdfSharpCore	Appends uploaded PDFs and draws PNG and JPG onto pages.
Files	Local disk, behind `IFileStorageService`	Bytes stay out of SQL Server. A later Azure Blob store can be a second implementation of the same interface.
Share	An Outlook `.eml` file	The engineer sends it locally. SMTP is not connected.
Health	`GET /health`	Confirms the API can reach SQL Server.

Addresses a developer uses

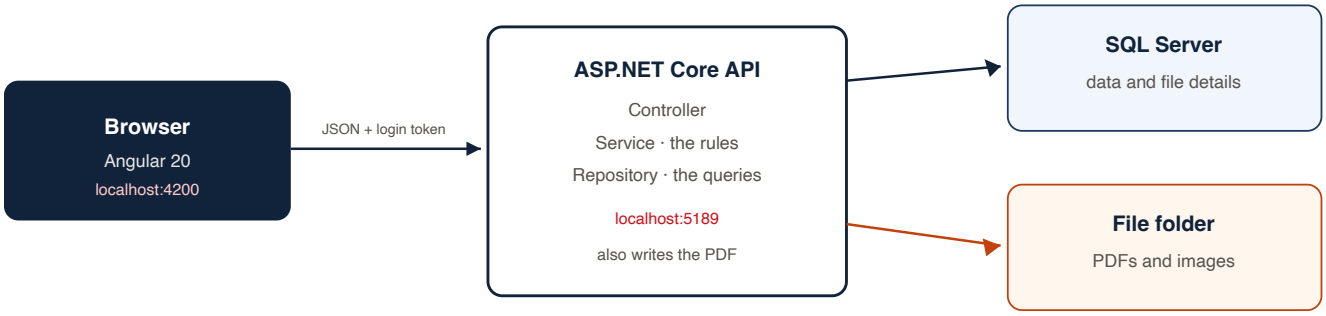
Piece	Address
Website	http://localhost:4200
API	http://localhost:5189
Health	http://localhost:5189/health
SQL Server	localhost, port 1433, database cigt, Docker volume cigt-sql-data

The website calls `environment.apiUrl`. In development that is the API above. In a production build it is `/api` on the same site, so a reverse proxy or IIS can host both. Production secrets (database password,

JWT signing key, upload folder, allowed website origin) come from environment variables. They are empty in the committed production file on purpose.

7. How the system is wired

Three processes run together. The browser never opens SQL Server. The browser never writes the upload folder.



What happens on one save

1. The Angular screen calls an Angular service. The service uses the API address from configuration. It does not hard-code a server name.
2. An interceptor attaches the login token. Each screen does not do that itself.
3. The controller checks that the person is signed in, calls the service, and returns the standard envelope.
4. The service checks the rules (required name, “this record belongs to you,” and so on).
5. The repository reads or writes SQL Server with Entity Framework Core.
6. If the action is an upload or a PDF, the service also calls the file-storage interface. The database row and the file are separate.

A normal success looks like `{ success, message, data }`. A failure looks like `{ success: false, message, errors }`. Controllers return that wrapper. They do not return raw database rows to the browser.

A sensible first production shape

One API instance and one SQL Server are enough for the Implementation team. This tool does not need a cache, a queue, or Kubernetes for a first release. It cannot run as two API servers until the files move off a single disk.

```
Browser                    The engineer or the customer
  HTTPS                      IIS or a reverse proxy
    Angular files              The website
    API                        ASP.NET Core, one instance to start
      SQL Server                 With backups
      File store                 Azure Blob, or one disk that is backed up
```

8. Folder structure

The repository is two applications plus a few documents. If a change is a screen, it belongs under `frontend`. If a change is a rule, a table, or a file, it belongs under `API`. The website does not contain database code.

```
frontend/                 The website
  src/app/                  All screens, and the code that calls the API
API/                      The server. The only part that talks to the database and the files
  Controllers/              The HTTP door
  Services/                 Business rules, PDF, login, files
  Repositories/             SQL queries
  Models/                   The tables, as C# classes
  DTOs/                     The shapes sent to the website
  Data/                     Database setup and starter data
  Migrations/               The history of table changes
  Configuration/            Settings objects: login, storage
  Assets/                   The ARCON logo used on the PDF
  Uploads/                  Real files on a developer machine. Not committed to git
  Program.cs                Starts the API and wires the pieces together
docker-compose.yml        SQL Server for development
docs/                     This review, and the longer risk brief
README.md                 How a person runs the application
DEMO.md                   What to click in a demo
```

Website folders, and the reason each one exists

Folder	What a person finds there
`auth/`	Sign in, register, forgot password, verify email.
`layouts/`	The frame around pages. The signed-in frame has the top bar. The signed-out frame is the login page.
`dashboard/`	Home after login. Search across this user's work.
`clients/`	Create a client and open a client.
`projects/`	Create an application, fill the questionnaire, review, and version history.
`customer-form/`	The public form. It sits outside the signed-in frame.
`users/`	The Admin page for accounts. Hidden unless the signed-in person is an Admin.
`settings/`	The signed-in person's own settings.
`services/`	The HTTP calls: auth, clients, projects, files, PDF, users. Screens use these. They do not call the API ad hoc.
`models/`	The TypeScript shapes of a client, a project, a user, an attachment.
`guards/`	Who may open a route. Signed-in pages, signed-out pages, and the Admin users page.
`interceptors/`	Attach the token. Show the loading state.
`validators/`	Email must be @arconnet.com. Password rules: 8 characters, upper, lower, and a digit.
`shared/components/`	Pieces used on many screens: search, dialogs, cards, status, empty and not-found states.

8. Folder structure, continued

Server folders

Folder	The rule for it
`Controllers/`	HTTP only. Call a service. Return the standard envelope. Do not put business rules here.
`Services/`	The rules: who owns the record, how a PDF is built, how a file is saved, how a user is created.
`Repositories/`	Entity Framework queries. No HTTP, and no “should this be allowed” decisions.
`Models/`	One class per table the product owns, plus the Identity user.
`DTOs/`	Request and response types. The browser never receives an entity class.
`Data/`	`ApplicationDbContext` maps classes to tables. `DbSeeder` creates roles, a demo admin, and sample clients only when the client table is empty.
`Migrations/`	Generated by Entity Framework. Applied when the API starts, which is right for a laptop and risky for production (two servers can start a migration at once).
`Uploads/`	Developer files. Git ignores this folder. Production config currently points at `C:\inetpub\cigt\Uploads`, which only works on one Windows server.

Screens and addresses

Address	Who	Screen
`/login` `/register`	Signed out	Sign in and create an Engineer account.
`/forgot-password` `/verify-email`	Signed out	Present on screen. Mail is not sent.
`/dashboard`	Signed in	Home and search.
`/clients/new` `/clients/:id`	Signed in	New client, client detail.
`/project/new`	Signed in	New application.
`/project/:id`	Signed in	Review, versions, PDF, share.
`/project/:id/edit`	Signed in	The questionnaire workspace.
`/users`	Admin	List and create accounts. Others are turned away.
`/settings`	Signed in	Own settings.
`/form/:token`	Anyone with the link	Customer form, then a success page.

API groups

Route	Purpose	Login
`api/auth`	Sign in, register, refresh, sign out, profile, change password, the email placeholders.	Mixed. Profile needs a token.
`api/clients`	List, create, delete this user’s clients.	Signed in
`api/projects`	List, create, update, delete, generate the customer link.	Signed in
`api/attachments`	List, upload a real file, delete. An older metadata-only upload still exists and should not be used for new work.	Signed in

Route	Purpose	Login
`api/projects/{id}/generate-pdf`	Build the PDF.	Signed in
`api/projects/{id}/share-email`	Build the Outlook draft.	Signed in
`api/customer-form`	Read and submit by token. Signed-in user can read responses for a project they own.	Public for the token routes
`api/users`	List and create accounts.	Admin, otherwise 403

9. What is stored

SQL Server holds structured data. It does not hold the PDF or the screenshots.

```
Users and roles          Identity tables and refresh tokens
Clients                  Company, contact, industry, country. OwnerUserId is the ARCON user who
                         created it
  Projects                 Application name, status, priority, engineer, public form token.
                           OwnerUserId works the same way
    CustomerForms            Customer answers, stored as JSON text
    Attachments              File name, type, size, path, who uploaded it. Not the file bytes
    ProjectDocument          One document record per application
      DocumentVersions         Each save: answers, file list, completion percent
```

Record	Holds
Client	Company name, industry, primary contact, email, phone, country, address, notes, owner, timestamps.
Project	Name, client, application name, description, engineer, priority, expected date, completion percent, status, form token, owner.
Project status	Draft, In Progress, Pending Review, Completed.
Application types	Used by the questionnaire: LDAP, database, API, SDK, RPA.
Customer form	The token and the submitted answers as JSON text.
Attachment	File name, content type, size, relative path, uploader, time. No file bytes.
Document version	Version number, save type, answers JSON, attachment list JSON, completion percent.
Lookup items	Seeded lists: status, priority, application type, role names.

> **UAT username and UAT password** are fields in the questionnaire. They are stored as plain JSON and they are printed in the PDF. That is acceptable for a demo. It is a decision for security before any real customer project is saved: encrypt those fields, or remove them and keep credentials in the existing secrets process.

Paths on disk follow a fixed pattern. Uploads: `attachments/{projectId}/{id}_{fileName}`. Generated PDFs: `generated/{projectId}/latest.pdf` plus a timestamped copy. The upload request cap is about 100 MB.

10. Files, the PDF, and email

Why the files are not inside SQL Server

The database stores the catalogue. The folder stores the bytes. That split is deliberate.

- A year of connector packs and screenshots would turn a small database into a very large one.
- Every backup would copy every file again. Restores would get slow.
- Opening a project list would drag large files into memory with the rows.
- A 50 MB upload inside a database transaction grows the transaction log.
- Two API servers can share one database. They cannot share a file that was written onto only one server's disk.

The code already has one storage interface. Today the only provider is `Local`. Any other provider name stops the API on startup, so production cannot be pointed at a cloud setting that does not exist yet. The recommended production move is Azure Blob (or ARCON's standard file store) as a second implementation of that same interface. Upload, delete, and PDF generation should keep calling the interface.

If a listed file is missing, PDF generation skips that file and still produces the document. That is kind to a demo and quiet in production: a lost disk looks like a PDF with holes, not like an error.

How the PDF is built

1. Load the project, the saved answers, and the attachment list.
2. QuestPDF draws the ARCON cover and the sections. A questionnaire field appears in the PDF only if it is listed in the field catalogue in the PDF service. Adding a question on the screen is not enough.
3. PdfSharpCore appends uploaded PDFs and places PNG and JPG on their own pages. Other file types can be stored and are not merged.
4. The result is written under the generated folder as `latest.pdf` and as a timestamped copy.

> QuestPDF is running under the Community license. ARCON needs the commercial license confirmed before a production PDF is sent to a customer.

Email

Share builds a draft file. It does not connect to Microsoft 365 or SMTP. There is no record that the connector team received anything. If Outlook is not on the engineer's machine, the draft cannot be sent from CIGT. Whether a draft is enough for version 1 is a leadership decision.

11. The questionnaire

These are the sections an engineer fills, and the sections the PDF prints. This is the content of the old Word document, made into fields.

Section	What it asks
About the application	Purpose, whether it is the source of truth, UAT environment (server, username, password), application type, how we connect, legacy or web.
Application integration	User onboarding, modification, deletion (soft or hard), reactivation, SSO type, reconciliation strategy, default entitlement, entitlement types.
Converged Identity	Which CI package, how the application relates to CI once integrated, module diagram notes.
Source of truth	Onboarding strategy, onboarding scan, the attributes that will be used.
Encryption	Which user fields are encrypted, whether API payloads are encrypted, encoded fields, the algorithm.
General information	Link to the API documentation.
Special comments	Anything that does not fit the fields above.

Empty answers are left out of the PDF. Filled answers are printed with a human label, not the internal field name.

12. Login, roles, and visibility

Login

Identity stores the user. The API issues a JWT access token and a refresh token. The website sends the access token on later calls.

Access token: 60 minutes in production settings, 120 in development. Refresh token: 7 days.

Lockout: 5 failures, then 15 minutes.

Roles

Three roles only: Admin, Engineer, Viewer. Do not add more without a decision.

Public registration cannot create an Admin. The Users API checks the Admin role and returns 403 otherwise.

Other APIs check only “is this person signed in?” A Viewer can still call edit, delete, upload, and PDF.

Who sees which clients

Every client and every project stores `OwnerUserId`. List, open, update, and delete filter on that id. An engineer cannot open a colleague’s application by guessing the id. The public form is the exception: it is opened by token, not by owner. The engineer who owns the project can read the customer’s submitted answers.

Sample clients are inserted only when the client table is empty, and they are owned by the seeded admin. If a production database were emptied, the next API start would insert those sample banks again. Production should seed roles only, and the first admin should be created outside the demo path.

What is finished

Area	State
Sign in, refresh, lockout, register, profile	Done Public registration is always Engineer. Real email verification is not done.
Users page	Done List and create. This is not a full admin console.
Dashboard, clients, projects, questionnaire, versions	Done
PDF, merged files, images on their own pages	Done
Outlook draft	Done Sending mail from the server is not.
SQL Server schema, file details in SQL, bytes on disk	Done
Each user sees only their own clients and projects	Done
Cloud file storage	Not built The interface is ready.
SMTP, audit log, Viewer blocked from edit	Not built
Word and PowerPoint	Removed PDF is the only export.

13. Decisions before the bigger design

These are the items to settle in the technical review. They change the use cases. They do not require a new architecture.

Decision	Why it matters before UML
Where production runs	One Windows server with IIS, or Azure (App Service, Azure SQL, Blob). The file use case depends on this.
Where files live	Blob storage is the recommended design. A single backed-up disk is acceptable only with one API instance, written down as a limit.
UAT passwords	If they stay, the data model gains encryption and tighter access. If they go, those two fields leave the questionnaire and the PDF.
Who may have an account	Open Engineer registration for any @arconnet.com address is still on. IT can provision everyone instead. Admin creation already happens only on the Users page.
What Viewer means	Today Viewer is a label. If the role must be read-only, that is a new rule on edit, delete, upload, and share. It is not a new application.
Whether Admin sees all work	Today Admin does not. If leadership wants a cross-team view, that is a new use case. The owner column can stay.
Email	An Outlook draft may be enough for version 1. Server-side send, plus a record that it was sent, is a new use case and needs SMTP or Microsoft 365.
QuestPDF license	Procurement confirms the commercial license before the first customer PDF.
Customer link	Treat the link as a secret. Expiry, or a one-time check, should be decided before links go to customers.

Close these before real customer data

- Files on one server’s disk. A redeploy or a second server loses the screenshots while the database still lists them.
- Demo admin password and sample clients created on startup when the client table is empty.
- Database password and JWT signing key must come from a secret store, and must differ from development.
- HTTPS in front of the site. Development is plain HTTP on localhost.
- Role checks on edit, delete, and share, once Viewer and Admin mean what leadership decides.
- Real email, or hide the verify and reset screens until mail exists.
- A decision on UAT credentials in the JSON and in the PDF.
- Run database migrations as a release step, then start the API. Do not rely on two servers migrating themselves at startup.

Also worth knowing, and not blockers for the design workshop: there is no malware scan, no audit log, no automated test suite around login, upload, and PDF, and generated PDFs accumulate because each run keeps a timestamped copy as well as `latest.pdf`.

14. How to start the formal UML

The workshop can start from this pack. It does not need to rediscover the product.

Keep

- Actors: Engineer, Admin, Viewer, Customer, Connector team.
- Use cases UC-01 to UC-18 as the baseline.
- Angular, ASP.NET Core, SQL Server, files outside the database, PDF only.
- The layering: screen, service call, controller, service, repository.
- One owner on each client and project.
- The seven questionnaire sections as the document content.

Add only after a decision

- Read-only Viewer.
- Admin view of every engineer's work.
- Server-sent email and an audit log.
- Blob storage as the file location.
- Expiry on the customer link.
- Encryption or removal of UAT passwords.

Suggested order for the diagrams

1. **Use case diagram.** Start from section 3. Add ovals only for the decisions in section 13. Do not draw a second system for PDF or for files. Those are inside CIGT.
2. **One activity or sequence for the main journey** in section 5: sign in, client, application, save, upload, PDF, share. A second sequence for the customer link.
3. **A data diagram** from section 9. Show Attachment as metadata with a path. Do not draw a binary column for the file.
4. **A component diagram** from section 7: Browser, API, SQL Server, file store. Classes inside the API follow Controller, Service, Repository.

> None of the production work requires a rewrite. The product shape — Angular, ASP.NET Core, SQL Server, PDF, and files beside the database — is the one to keep. The longer risk write-up for the same system is `docs/CIGT-Technical-Brief.md`. This PDF is the picture to review first.